Humanloop is joining Anthropic View the announcement

Explainer 2025/02/01

8 Retrieval Augmented Generation (RAG) Architectures You Should Know in 2025

By Conor Kelly
Growth

Retrieval Augmented Generation (RAG) is a technique in large language models (LLMs) that enhances text generation by incorporating real-time data retrieval. Unlike traditional models that rely solely on their pre-trained knowledge, RAG allows models to search external databases or documents during the generation process, resulting in more accurate and up-to-date responses. By blending retrieval and generation, RAG architecture addresses key limitations such as hallucinations (where models generate inaccurate or fabricated information) and improves fact-based, contextually relevant outputs.

In RAG, the model operates in two stages:

- **Retrieval**: It first fetches relevant data from external sources based on the query or prompt.

- **Generation**: The model then processes the retrieved data and generates a coherent and informed response.

This dynamic approach makes RAG highly effective for use cases requiring real-time information retrieval, such as open-domain question answering, customer support automation, or enterprise search. Additionally, RAG offers advantages over techniques like fine-tuning, where models are retrained on new data, or prompt engineering, which focuses on optimising the input query for better results.

There are many forms of architecture which RAG can take. This article dives into the 8 most popular RAG architectures you should know if you're building generative AI applications:

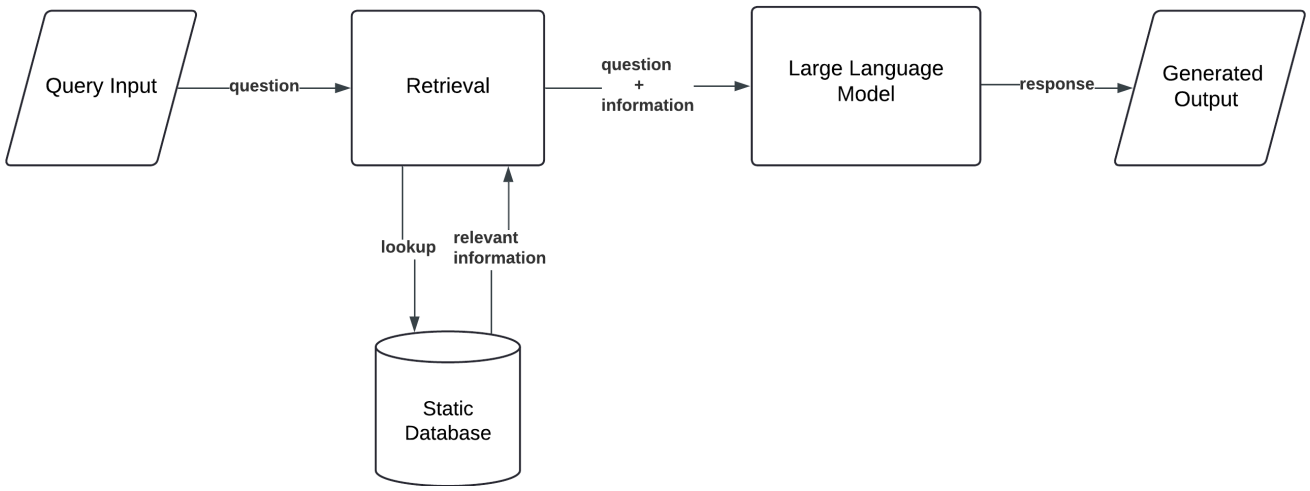
1. Simple RAG

This is the most basic form of RAG. In this configuration, the language model retrieves relevant documents from a static database in response to a query, and then generates an output based on the retrieved information. This straightforward implementation works well in situations where the database is relatively small and doesn't require complex handling of large or dynamic datasets.

Workflow:

- **Query Input**: The user provides a prompt or question.
- **Document Retrieval**: The model searches a fixed database to find relevant information.
- **Generation**: Based on the retrieved documents, the model generates a response that is grounded in the real-world data it found.

Use Case: Simple RAG is ideal for FAQ systems or customer support bots where responses need to be factually accurate, but the scope of information is limited to a known set of documents, like a product manual or knowledge base.



Simple RAG

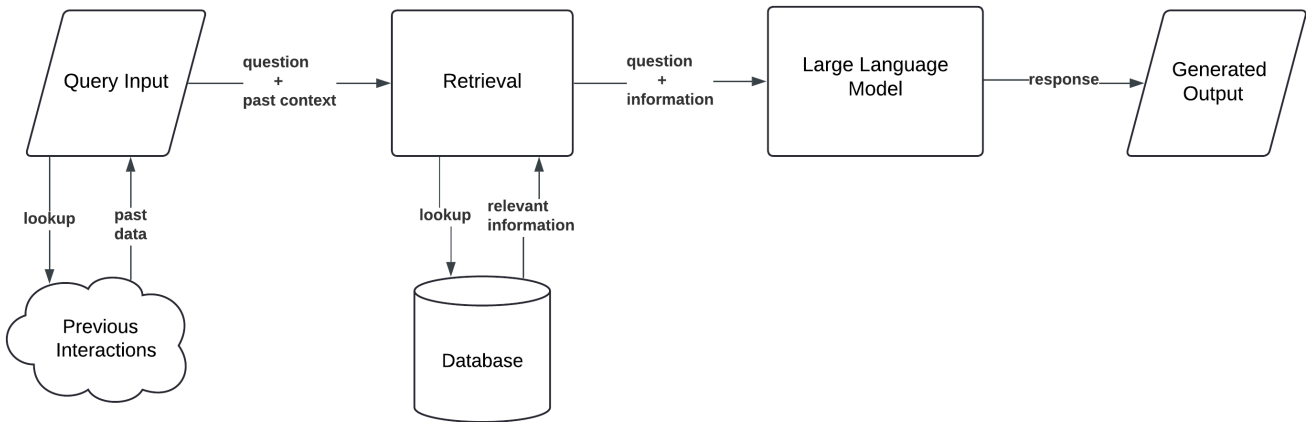
2. Simple RAG with Memory

Simple RAG with Memory introduces a storage component that allows the model to retain information from previous interactions. This addition makes it more powerful for continuous conversations or tasks that require contextual awareness across multiple queries. **Prompt caching** can be used with Simple RAG to achieve this.

Workflow:

- **Query Input**: The user submits a query or prompt.
- **Memory Access**: The model retrieves past interactions or data stored in its memory.
- **Document Retrieval**: It searches the external database for new relevant information.
- **Generation**: The model generates a response by combining retrieved documents with the stored memory.

Use Case: This implementation is particularly useful for chatbots in customer service, where ongoing interactions require the model to remember user preferences or prior issues. It's also beneficial in personalised recommendations where historical data improves the relevance of responses.



Simple RAG with Memory

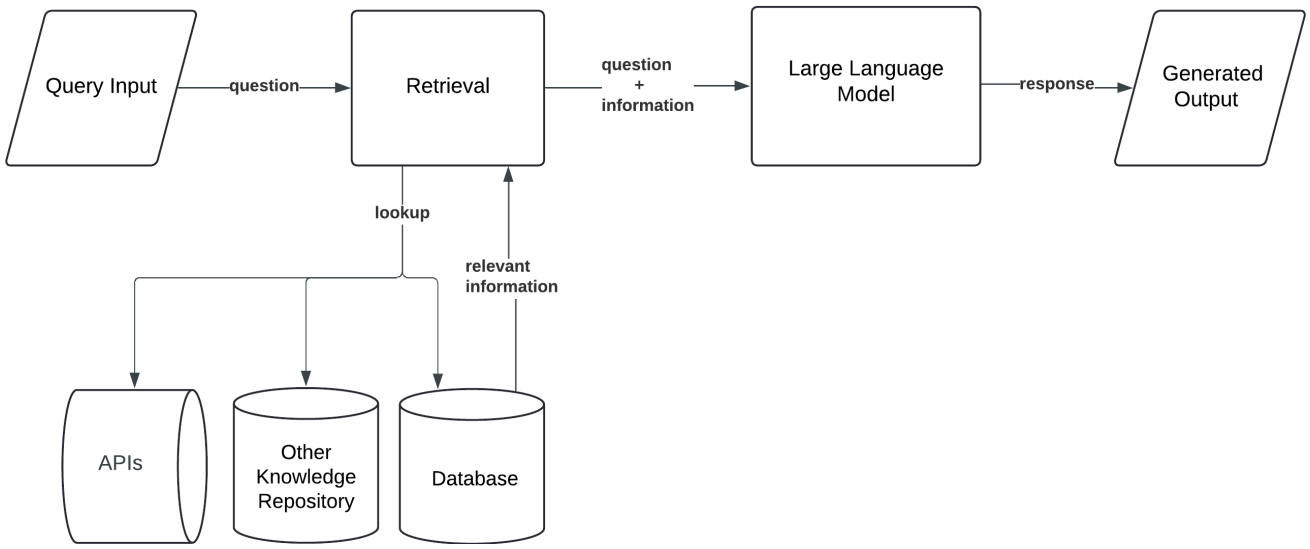
3. Branched RAG

Branched RAG enables a more flexible and efficient approach to data retrieval by determining which specific data sources should be queried based on the input. Instead of querying all available sources, Branched RAG evaluates the query and selects the most relevant source(s) to retrieve information.

Workflow:

- **Query Input**: The user submits a prompt.
- **Branch Selection**: The model evaluates multiple retrieval sources and selects the most relevant one based on the query.
- **Single Retrieval**: The model retrieves documents from the selected source.
- **Generation**: The model generates a response based on the retrieved information from the chosen source.

Use Case: Branched RAG is ideal for complex queries requiring specialized knowledge, such as legal tools, or multidisciplinary research, where the model needs to choose the best information source without consolidating irrelevant data from multiple sources.



Branched RAG

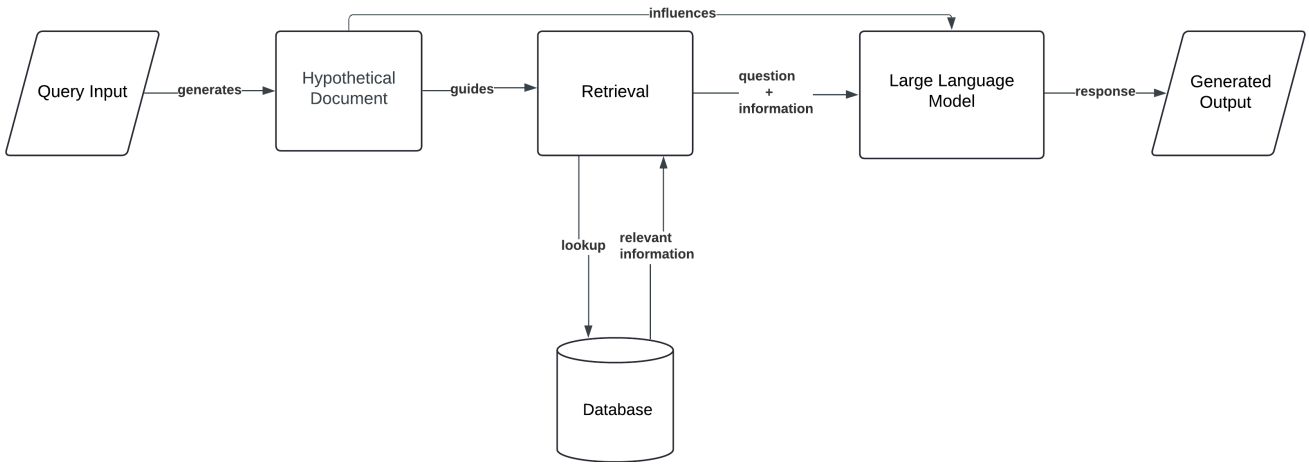
4. HyDe (Hypothetical Document Embedding)

HyDe (Hypothetical Document Embedding) is a unique RAG variant that generates hypothetical documents based on the query before retrieving relevant information. Instead of directly retrieving documents from a database, HyDe first creates an embedded representation of what an ideal document might look like, given the query. It then uses this hypothetical document to guide retrieval, improving the relevance and quality of the results.

Workflow:

- **Query Input**: The user provides a prompt or question.
- **Hypothetical Document Creation**: The model generates an embedded representation of an ideal response.
- **Document Retrieval**: Using the hypothetical document, the model retrieves actual documents from a knowledge base.
- **Generation**: The model generates an output based on the retrieved documents, influenced by the hypothetical document.

Use Case: HyDe is especially useful for research and development, where queries may be vague, and retrieving data based on ideal or hypothetical responses helps refine complex answers. It also applies to creative content generation when more flexible, imaginative outputs are needed.



HyDe (Hypothetical Document Embedding)

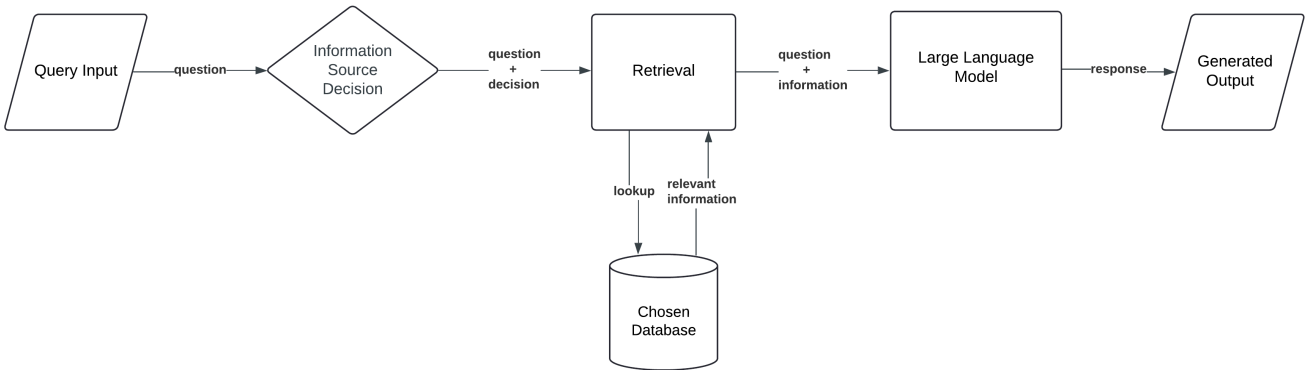
5. Adaptive RAG

Adaptive RAG is a dynamic implementation that adjusts its retrieval strategy based on the complexity or nature of the query. Unlike static models, which follow a single retrieval path regardless of the query, Adaptive RAG can alter its approach in real-time. For simple queries, it might retrieve documents from a single source, while for more complex queries, it may access multiple data sources or employ more sophisticated retrieval techniques.

Workflow:

- **Query Input**: The user submits a prompt.
- **Adaptive Retrieval**: Based on the query's complexity, the model decides whether to retrieve documents from one or multiple sources, or to adjust the retrieval method.
- **Generation**: The model processes the retrieved information and generates a tailored response, optimising the retrieval process for each specific query.

Use Case: Adaptive RAG is useful for enterprise search systems, where the nature of the queries can vary significantly. It ensures both simple and complex queries are handled efficiently, providing the best balance of speed and depth.



Adaptive RAG

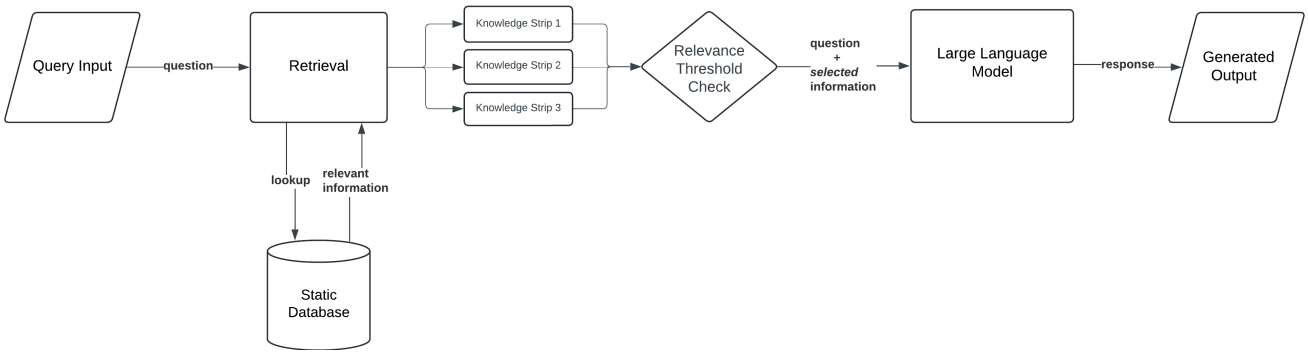
6. Corrective RAG (CRAG)

Corrective RAG (CRAG) incorporates a self-reflection or self-grading mechanism on retrieved documents to improve the accuracy and relevance of generated responses. Unlike traditional RAG models, CRAG critically evaluates the quality of the information retrieved before moving to the generation phase. The system breaks down retrieved documents into "knowledge strips" and grades each strip for relevance. If the initial retrieval fails to meet a threshold of relevance, CRAG initiates additional retrieval steps, such as web searches, to ensure it has the best possible information for generating the output.

Workflow:

- **Query Input**: The user submits a query or prompt.
- **Document Retrieval**: The model retrieves documents from the knowledge base and evaluates their relevance.
- **Knowledge Stripping and Grading**: The retrieved documents are broken down into "knowledge strips" — smaller sections of information. Each strip is graded based on relevance.
- **Knowledge Refinement**: Irrelevant strips are filtered out. If no strip meets the relevance threshold, the model seeks additional information, often using web searches to supplement retrieval.
- **Generation**: Once a satisfactory set of knowledge strips is obtained, the model generates a final response based on the most relevant and accurate information.

Use Case: Corrective RAG is ideal for applications requiring high factual accuracy, such as legal document generation, medical diagnosis support, or financial analysis, where even minor inaccuracies can lead to significant consequences. When **evaluating models like CRAG**, it's essential to assess both the accuracy of retrieval and the consistency of error correction to ensure optimal performance in sensitive domains.



Corrective RAG (CRAG)

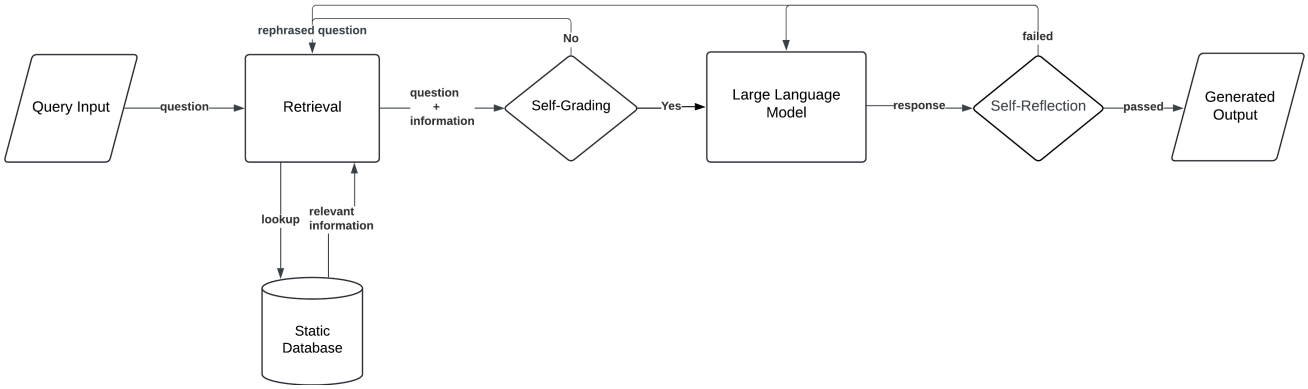
7. Self-RAG

Self-RAG introduces a self-retrieval mechanism, allowing the model to autonomously generate retrieval queries during the generation process. Unlike traditional RAG models, where retrieval is based solely on the user's input, Self-RAG can iteratively refine its retrieval queries as it generates content. This self-guided approach enhances the quality and relevance of information, especially for complex or evolving queries.

Workflow:

- **Query Input**: The user submits a prompt.
- **Initial Retrieval**: The model retrieves documents based on the user's query.
- **Self-Retrieval Loop**: During the generation process, the model identifies gaps in the information and issues new retrieval queries to find additional data.
- **Generation**: The model generates a final response, iteratively improving it by retrieving further documents as needed.

Use Case: Self-RAG is highly effective in exploratory research or long-form content creation, where the model needs to pull in more information dynamically as the response evolves, ensuring comprehensive and accurate results.



Self-RAG

8. Agentic RAG

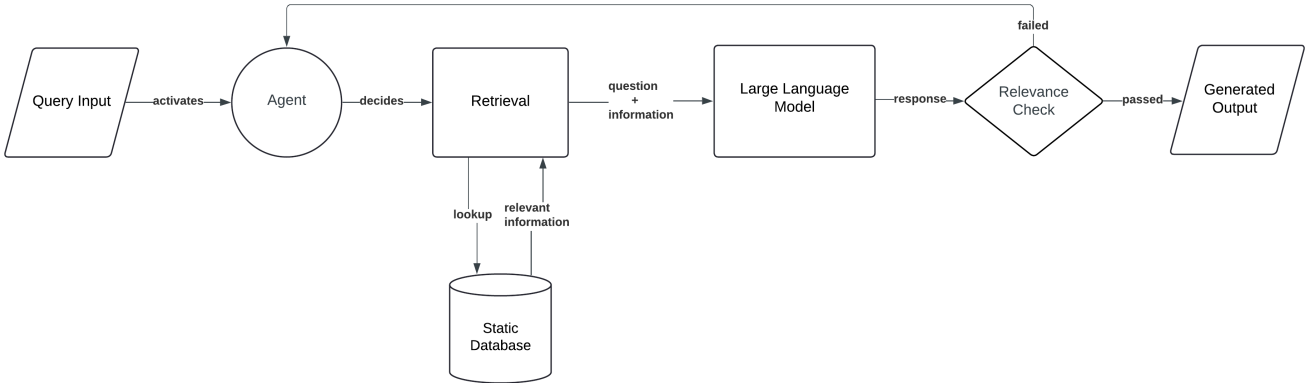
Agentic RAG introduces a more autonomous, agent-like behaviour in the retrieval and generation process. In this implementation, the model acts as an "agent" that can perform complex, multi-step tasks, proactively interacting with multiple data sources or APIs to gather information. What sets Agentic RAG apart is its ability to assign Document Agents to each individual document and orchestrate their interactions through a meta-agent. This system allows for more sophisticated decision-making, enabling the model to determine which retrieval strategies or external systems to engage with based on the complexity of the query.

Workflow:

- **Query Input**: The user submits a complex query or task.
- **Agent Activation**: The model activates multiple agents. Each Document Agent is responsible for a specific document, capable of answering questions and summarising information from that document.
- **Multi-step Retrieval**: The Meta-Agent manages and coordinates the interactions between the various Document Agents, ensuring the most relevant information is retrieved.

- **Synthesis and Generation**: The Meta-Agent integrates the outputs from the individual Document Agents and generates a comprehensive, coherent response based on the collective insights gathered.

Use Case: Agentic RAG is perfect for tasks like automated research, multi-source data aggregation, or executive decision support, where the model needs to autonomously pull together and synthesise information from various systems.



Agentic RAG

Learn more about building RAG applications

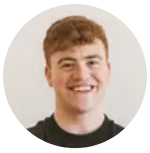
To learn more about RAG, check out our cookbook on **how to evaluate RAG** and our guide on **RAG explained**.

At Humanloop, we make it easy for enterprises to collaboratively develop and evaluate RAG-based AI applications. We provide all the tooling needed to manage prompts and retrievers and evaluate their performance to ensure your RAG application is reliable at scale.

If you're interested in learning how **Humanloop** can enable your enterprise to develop and evaluate RAG applications, **book a demo** to chat with our team.

Discuss on X

About the author



Conor Kelly

Growth

Conor Kelly is the Growth lead at Humanloop. He is an expert in generative AI application development and did graduate research on retrieval augmented generation (RAG) at UCL. Conor regularly appears on high-profile media outlets to discuss generative AI was nominated as Ireland's Young AI Role Model of the Year in 2023.

@conorkellyai LinkedIn

Ready to build successful AI products?

Book a 1:1 demo for a guided tour of the platform tailored to your organization.

Book a demo

Apollo.io

Humanloop



OpenAI Agents SDK ↗



LLM as a Judge ↗



Prompt Files ↗

Platform

LLM Evaluations

Prompt Management

AI Observability

Compliance and Security

Company

Careers 6

About

Trust Center

Newsletter

Your email

Subscribe

Resources

Blog

Pricing

Docs ↗

High Agency Podcast





© 2020 - 2045 Humanloop, Inc. Terms & Policies